AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2026

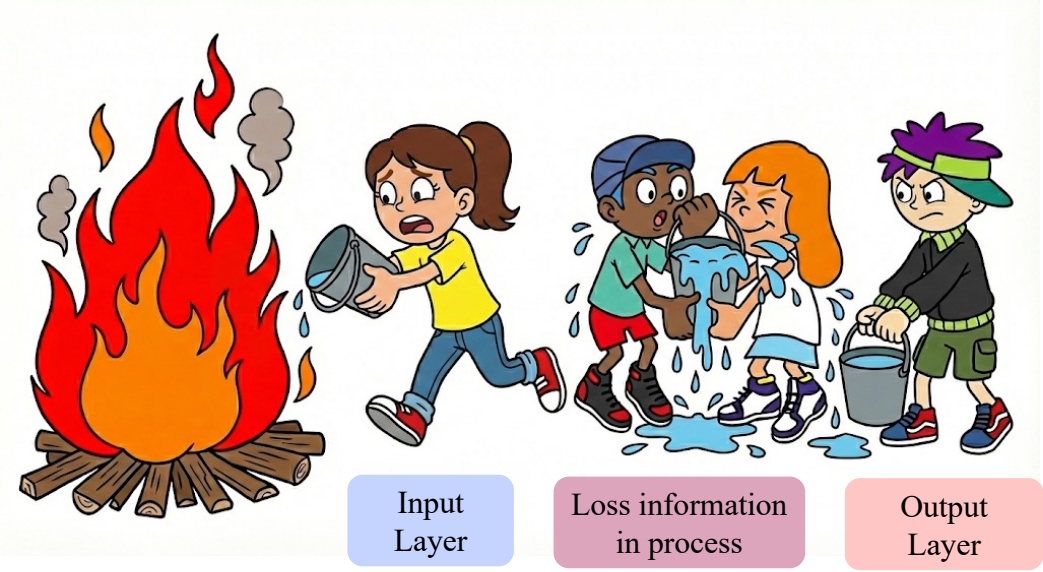
Tutorial: Vanishing Gradient

Nguyễn Thọ Anh Khoa
Dương Đình Thắng
Nguyễn Phúc Thịnh
Nguyễn Anh Khôi

I. Giới thiệu

Chúng ta thường hay nghĩ rằng, để mô hình tốt hơn, sẽ cần phải tăng số lượng tham số của mô hình lên. Thế nhưng điều này không phải lúc nào cũng đúng. Đến một ngưỡng nào đó, mạng của chúng ta sẽ gặp phải vấn đề "Vanishing Gradient" (tạm dịch: mất mát đạo hàm). Để hiểu bản chất của vanishing gradient, hãy tưởng tượng quá trình huấn luyện mạng nơ-ron giống như một trò chơi Team Building: "Chuyền nước tiếp sức".

Trong trò chơi này, một hàng người dài (các lớp mạng) phải chuyền tay nhau những xô nước từ cuối hàng về đầu hàng để dập lửa. Người cuối hàng (Output Layer) múc được một xô nước đầy (Gradient lớn). Tuy nhiên, mỗi chiếc xô đều bị thủng một lỗ nhỏ, khiến nước bị rò rỉ mỗi khi qua tay một người (tương ứng với việc nhân với đạo hàm nhỏ hơn 1). Nếu hàng người quá dài (Mạng quá sâu), lượng nước rò rỉ tích tụ qua từng người sẽ khiến chiếc xô trở nên cạn khô trước khi đến tay người đầu hàng (Input Layer). Kết quả là người đứng đầu không nhận được giọt nước nào để dập lửa (không thể cập nhật trọng số). Hiện tượng "cạn nước" này chính là Vanishing Gradient.

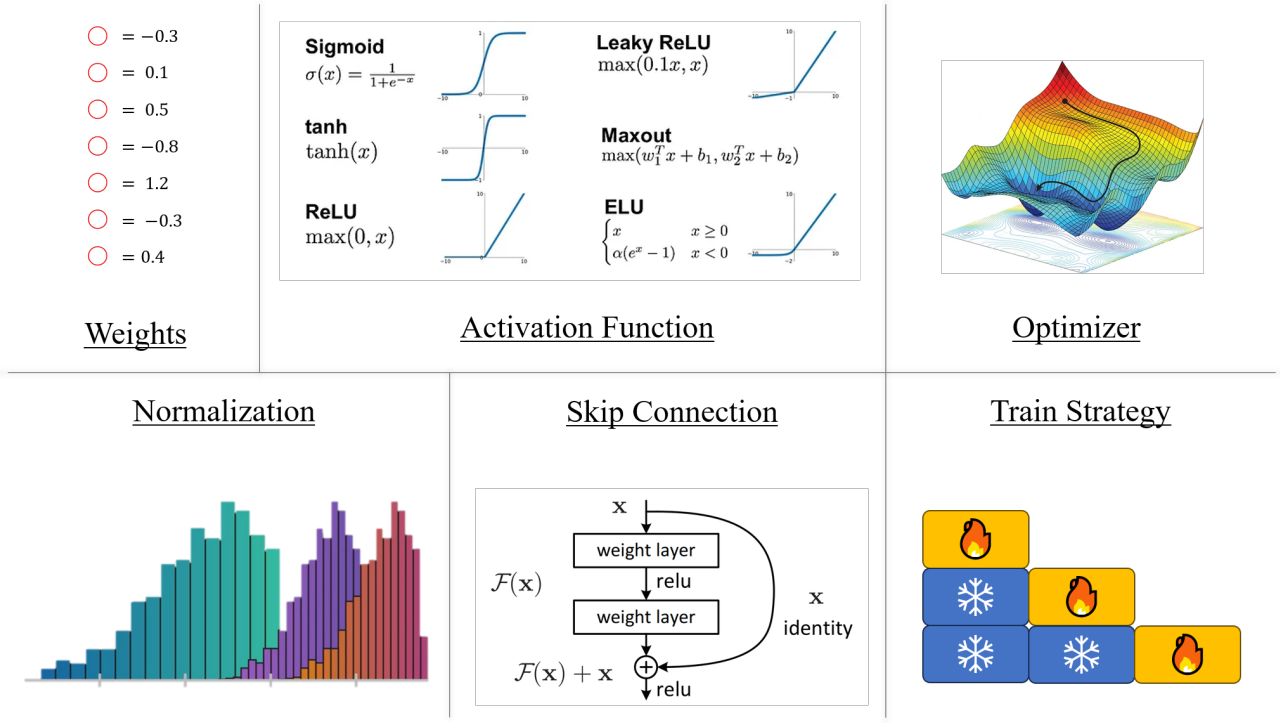


Hình 1: Ẩn dụ: Nước (Gradient) đại diện cho tín hiệu sửa lỗi. Khi đi qua mỗi người (Layer), nước bị rò rỉ một phần (Gradient bị nhân với số nhỏ < 1). Với mạng quá sâu, người đầu hàng nhận được chiếc xô rỗng.

Vanishing Gradient thường được nhận diện qua một số dấu hiệu điển hình:

- Trọng số của các layer gần output layer thay đổi đáng kể, trong khi trọng số của các layer gần input layer thay đổi rất ít hoặc hầu như không thay đổi.
- Trọng số có thể dần tiệm cận 0 trong quá trình huấn luyện, làm giảm hiệu quả cập nhật.
- Mô hình học rất chậm, và quá trình huấn luyện có thể bị đình trệ từ rất sớm, thường chỉ sau vài epoch đầu tiên.
- Phân phối của trọng số tập trung quanh giá trị 0, hạn chế khả năng truyền tải thông tin qua các lớp mạng.

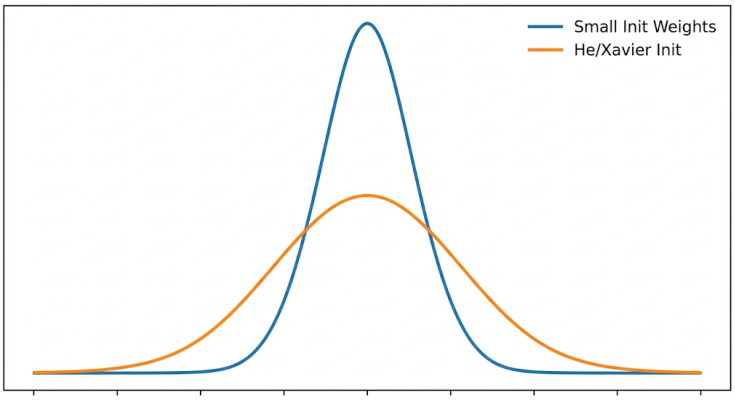
Nguyên nhân chính của hiện tượng Vanishing Gradient xuất phát từ quy tắc chuỗi (chain rule) trong quá trình lan truyền ngược. Gradient tại các layer gần input layer giảm dần theo tích gradient qua các lớp trước đó, dẫn đến gần như bằng 0 trong các mạng sâu. Đây là một trong những thách thức lớn nhất trong huấn luyện một deep neural network, đòi hỏi các phương pháp xử lý hiệu quả để khắc phục.



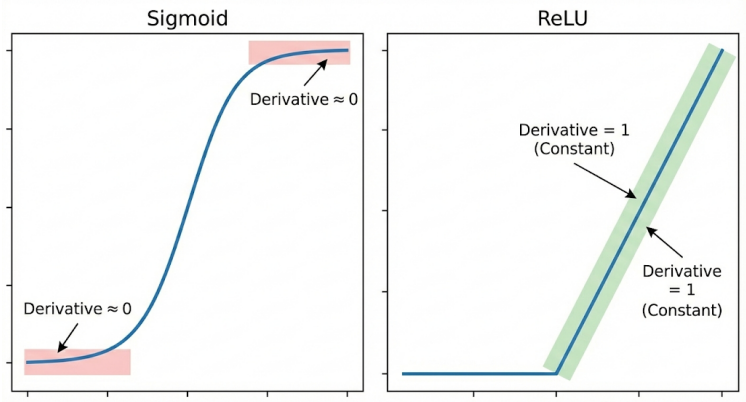
Hình 2: Một số phương pháp khắc phục Vanishing Gradient được đề cập trong project. Nguồn ảnh: Activation Function, Optimizer, Normalization, Skip Connection.

Đi vào kỹ các vấn đề thường gặp gây ra mất mát của đạo hàm, chúng ta có 6 ý chính được liệt kê như sau:

1. Weight Initialization (Khởi tạo trọng số): Kỹ thuật này tập trung vào việc khởi tạo trọng số ban đầu thích hợp để hạn chế vanishing gradient. Tại đây, chúng ta triển khai ý tưởng đơn giản bằng cách khởi tạo trọng số lớn với phân phối chuẩn (normal distribution). Ngoài ra có thể tham khảo thêm các phương pháp khởi tạo chuẩn như He Initialization hoặc Xavier để đảm bảo phương sai của đầu ra tương đồng với đầu vào.



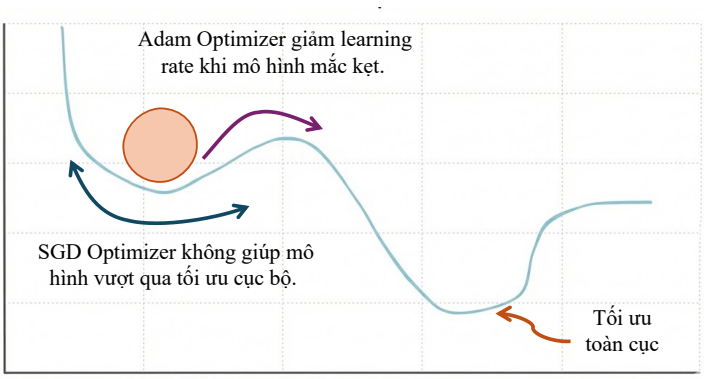
Hình 3: So sánh phân phối trọng số. Phân phối hẹp (xanh biển) khiến tín hiệu bị triệt tiêu, trong khi phân phối rộng hơn (cam) giúp duy trì dòng chảy thông tin.



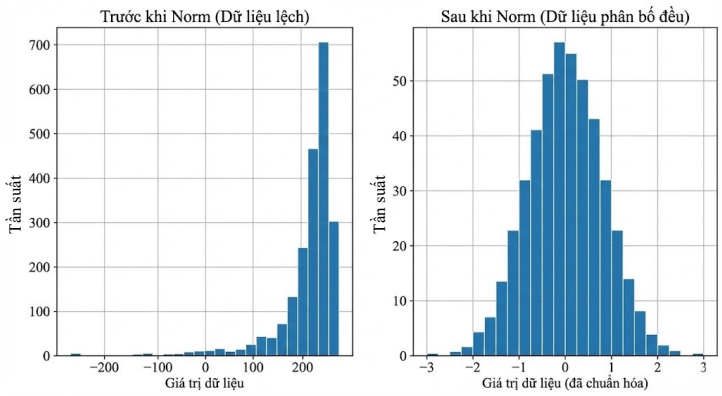
Hình 4: Hàm Sigmoid làm gradient nhỏ đi nhanh chóng do đạo hàm luôn nhỏ hơn 0.25. Ngược lại, ReLU duy trì đạo hàm bằng 1 với đầu vào dương.

2. Better Activation Function: Hàm kích hoạt Sigmoid truyền thống có đạo hàm nằm trong khoảng $[0, 0.25]$, khiến gradient bị nhân liên tiếp với các số nhỏ và triệt tiêu. Việc thay thế bằng các hàm hiện đại như **ReLU** (Rectified Linear Unit) hoặc Leaky ReLU giúp giải quyết vấn đề này vì đạo hàm của chúng là 1 (với miền dương), giúp gradient được bảo toàn khi lan truyền ngược. Các bạn có thể đọc thêm về tính chất đạo hàm của hai hàm trên ở đây.

3. Better Optimizer: Các thuật toán tối ưu như SGD thường gặp khó khăn trong một số trường hợp như thoát khỏi các vùng gradient nhỏ. Sử dụng các Optimizer hiện đại như **Adam** hoặc RMSProp giúp tự động điều chỉnh tốc độ học (learning rate) cho từng tham số, cho phép mô hình thoát khỏi các điểm tối ưu cục bộ và hội tụ nhanh hơn ngay cả khi gradient ban đầu nhỏ.



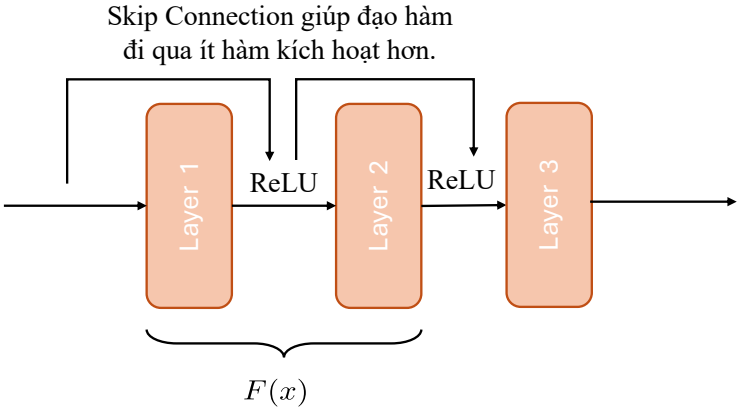
Hình 5: Minh họa bề mặt Loss Function. Optimizer thông minh (mũi tên tím) tìm đường xuống dốc hiệu quả hơn so với SGD thông thường.



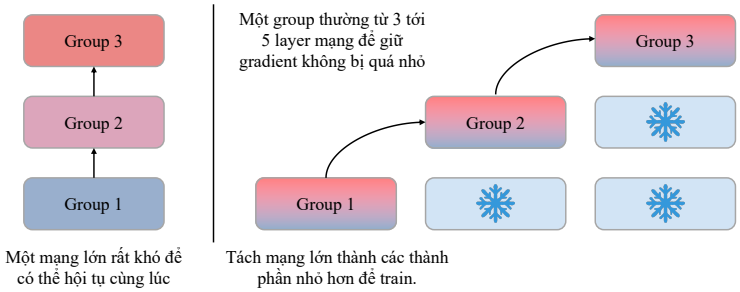
Hình 6: Batch Normalization chuẩn hóa dữ liệu đầu vào của lớp ẩn (đưa về phân phối chuẩn), giúp tránh rơi vào vùng bão hòa của hàm kích hoạt.

4. Normalize Inside Network: Hiện tượng "Internal Covariate Shift" xảy ra khi phân phối đầu vào của các lớp thay đổi liên tục trong quá trình train. Kỹ thuật **Batch Normalization** giúp chuẩn hóa đầu ra của lớp trước về dạng phân phối chuẩn (mean=0, std=1). Điều này giữ cho giá trị kích hoạt nằm trong vùng "nhạy" của hàm activation, ngăn chặn gradient bị triệt tiêu.

5. Skip Connection (ResNet): Trong các mạng rất sâu, việc truyền gradient qua quá nhiều lớp chồng lên nhau rất khó khăn. **Skip Connection** tạo ra một "đường tắt", cộng trực tiếp đầu vào x vào đầu ra của lớp $F(x)$. Điều này tạo ra một luồng gradient "siêu tốc" chảy ngược về các lớp đầu tiên mà không bị suy giảm cũng không bị ảnh hưởng bởi các hàm kích hoạt, cho phép huấn luyện các mạng sâu hàng trăm lớp.



Hình 7: Kiến trúc Residual Block. Đường cong (mũi tên) biểu diễn kết nối tắt x được cộng vào đầu ra, giúp gradient dễ dàng lan truyền ngược.



Hình 8: Chiến thuật huấn luyện phân tầng: Các lớp màu xanh (bông tuyết) được đóng băng trọng số, chỉ tập trung huấn luyện lớp mới (ngọn lửa).

6. Train Layers Separately (Fine-tuning): Thay vì huấn luyện cả một mạng sâu cùng lúc (khiến các lớp đầu khó học), ta có thể huấn luyện từng phần nhỏ. Bắt đầu với mạng nông, sau khi hội tụ thì đóng băng (freeze) các lớp này và thêm các lớp mới vào để huấn luyện tiếp. Cách này đảm bảo các lớp dưới học được đặc trưng tốt trước khi mô hình trở nên quá phức tạp.

Trong project này, chúng ta sẽ tìm hiểu vấn đề Vanishing Gradient trong mạng MLP và thử nghiệm các giải pháp cải tiến hiệu quả học của mô hình (bên cạnh 6 phương pháp trên chúng ta sẽ đọc thêm 3 kỹ thuật nâng cao). Tổng quan nội dung bao gồm:

1. Thiết lập một mạng MLP điển hình dễ bị Vanishing Gradient với các đặc điểm: hàm kích hoạt Sigmoid, độ sâu lớn, và cấu trúc không tối ưu để quan sát cách gradient giảm dần qua các lớp.

2. Thử nghiệm các giải pháp khác nhau để khắc phục Vanishing Gradient và đánh giá mức độ cải thiện. Các giải pháp bao gồm:

   (a) Weight increasing: Thay đổi cách khởi tạo trọng số ban đầu để tăng giá trị gradient, qua đó giảm thiểu khả năng gradient trở nên quá nhỏ.

   (b) Better activation: Thử nghiệm các hàm kích hoạt tiên tiến, không bị bão hòa dễ dàng như Sigmoid, nhằm giữ gradient ổn định hơn.

   (c) Better optimizer: Sử dụng các thuật toán tối ưu hiện đại hơn để tăng tốc độ hội tụ và giảm thiểu vấn đề Vanishing Gradient.

   (d) Normalize inside network: Áp dụng kỹ thuật chuẩn hóa để giữ giá trị đầu ra trong khoảng ổn định, từ đó giúp gradient không giảm quá nhanh qua các lớp.

   (e) Skip connection: Tích hợp các skip connection trong kiến trúc mạng để tạo đường dẫn gradient trực tiếp từ các lớp trước đến các lớp sau, như kiến trúc ResNet đã chứng minh hiệu quả.

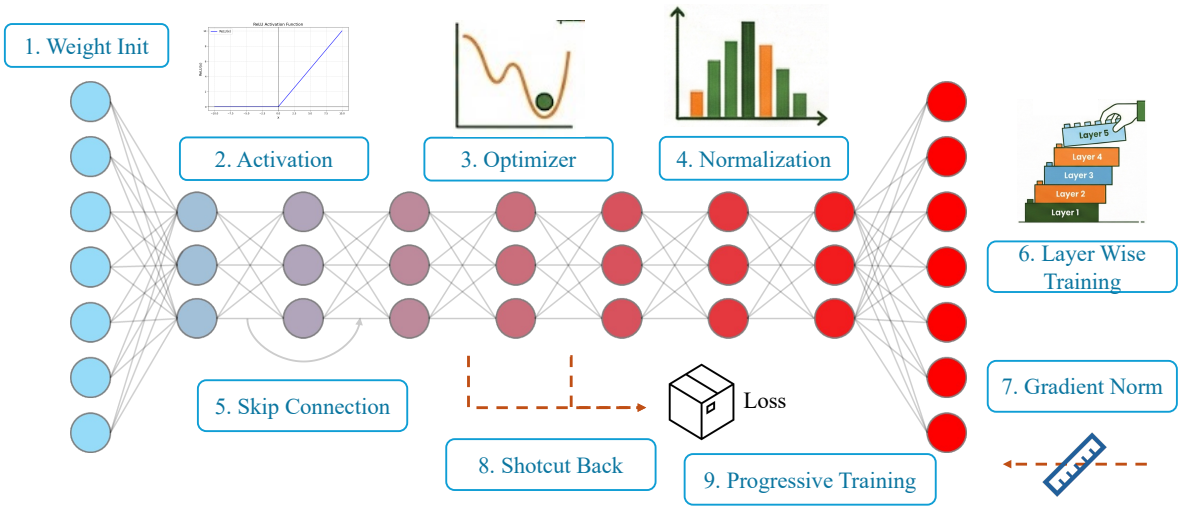
   (f) Train layers separately (Fine-tuning): Huấn luyện tuần tự các lớp để giảm thiểu tác động của các lớp sâu hơn đến gradient.

   (g) Gradient normalization: Chuẩn hóa gradient trong quá trình lan truyền ngược để duy trì giá trị gradient hợp lý.

   (h) Shortcut Back: Trong project này sử dụng Gradient rescaling lấy cảm hứng từ Shortcut Back. Backward hook nhân gradient đang truyền ngược qua mỗi tầng ẩn với hệ số phụ thuộc vào α.

   (i) Progressive Training: Với ý tưởng tương tự như ở ý (f), chúng ta sẽ train mô hình với các layer nông trước, rồi dần dần mở các layer sau qua từng epoch với learning rate cao hơn thông thường.

Pipeline của project có thể được minh họa như sau:



Hình 9: Pipeline minh hoạ tổng quan nội dung project.

Thông qua phân tích và so sánh kết quả từ các giải pháp, chúng ta sẽ rút ra điểm mạnh, yếu và tính ứng dụng của từng phương pháp, từ đó đề xuất các chiến lược phù hợp để khắc phục vấn đề Vanishing Gradient trong mạng MLP.

II. Cài đặt chương trình

Dựa trên mô tả project ở phần trước, bước đầu tiên ta sẽ tiến hành xây dựng chương trình huấn luyện và đánh giá một mô hình MLP trên bộ dữ liệu Fashion MNIST. Mô hình này sẽ được thiết kế với chủ đích thể hiện được vấn đề "Vanishing Gradient", và ta gọi mô hình này là baseline model.

II.1. Import các thư viện cần thiết:

Đầu tiên, chúng ta cần import các thư viện cần thiết cho việc xử lý dữ liệu, xây dựng mô hình, và các công cụ hỗ trợ. Các thư viện này bao gồm PyTorch cho việc cài đặt mô hình, cùng với torchvision để tải dữ liệu Fashion MNIST, và matplotlib để trực quan hóa.

Import thư viện

```
import random
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch import nn
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision.datasets import FashionMNIST
```

II.2. Xác định phần cứng và cố định tham số ngẫu nhiên:

Để đảm bảo kết quả mô hình huấn luyện được là như nhau khi bắt đầu chạy chương trình ở Google Colab, chúng ta cần cố định tham số ngẫu nhiên (seed) cho mọi thư viện liên quan. Ngoài ra, chúng ta cũng cần xác định phần cứng (GPU nếu khả dụng hoặc CPU) để tối ưu hóa quá trình huấn luyện.

Cấu hình thiết bị và Seed

```
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

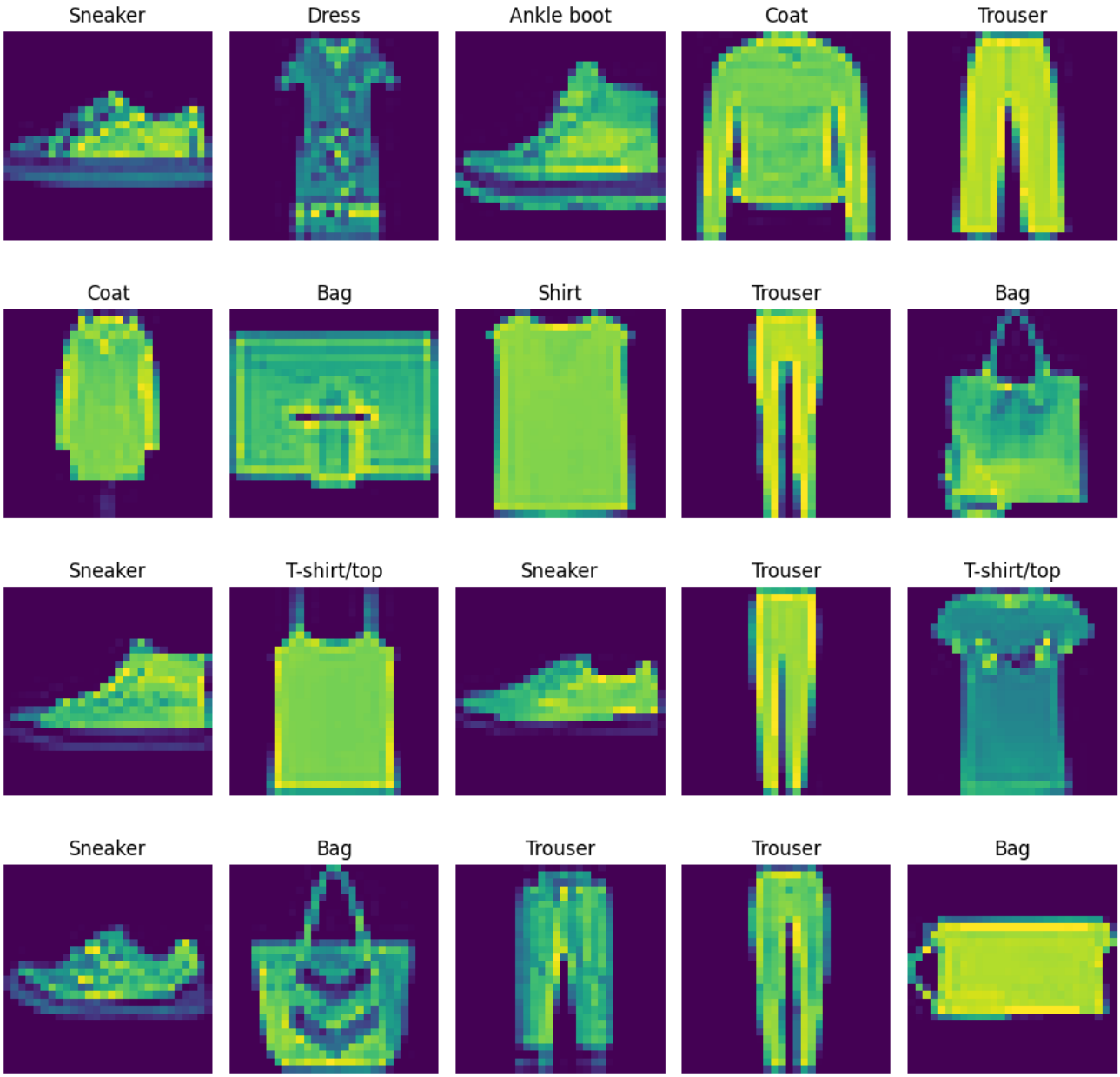
def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

```
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

SEED = 42
set_seed(SEED)
```

II.3. Tải bộ dữ liệu:

Bộ dữ liệu "Fashion MNIST" là một tập dữ liệu phổ biến, bao gồm các hình ảnh grayscale 28×28 của các loại quần áo, được sử dụng để phân loại 10 nhãn khác nhau. PyTorch cung cấp hàm tải tự động cho bộ dữ liệu này. Theo đó, chúng ta tải cả tập train và test, đồng thời chuyển đổi dữ liệu sang tensor.



Hình 10: Minh họa một vài mẫu dữ liệu từ bộ dữ liệu Fashion MNIST.

Tải dữ liệu

```
train_dataset = FashionMNIST('./data',
                             train=True,
                             download=True,
                             transform=transforms.ToTensor())
test_dataset = FashionMNIST('./data',
                            train=False,
                            download=True,
                            transform=transforms.ToTensor())
```

II.4. Chia bộ dữ liệu train/val/test:

Vì bộ dữ liệu gốc đã được chia sẵn tập train và tập test, chúng ta chỉ cần tạo thêm tập validation là đủ. Để tạo tập validation, ta chia bộ train ban đầu thành hai phần với tỉ lệ chia là 90% cho train và 10% cho validation. Các tập dữ liệu này sau đó được gói vào `DataLoader` để dễ dàng sử dụng trong quá trình huấn luyện.

Phân chia dữ liệu

```
train_ratio = 0.9
train_size = int(len(train_dataset) * train_ratio)
val_size = len(train_dataset) - train_size

train_subset, val_subset = random_split(train_dataset, [train_size, val_size])

train_loader = DataLoader(train_subset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_subset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print(f"Train size: {len(train_subset)}")
print(f"Validation size: {len(val_subset)}")
print(f"Test size: {len(test_dataset)}")
```

II.5. Xây dựng mô hình MLP:

Tiếp theo, chúng ta xây dựng mạng MLP với nhiều lớp ẩn. Mỗi lớp ẩn được nối tiếp bởi hàm kích hoạt Sigmoid. Sau khi định nghĩa kiến trúc mạng, chúng ta sẽ khởi tạo mô hình và khai báo hàm mất mát (CrossEntropyLoss) cùng thuật toán tối ưu (SGD).

Định nghĩa lớp MLP

```
class MLP(nn.Module):
    def __init__(self, input_dims, hidden_dims, output_dims):
        super(MLP, self).__init__()
        self.layer1 = nn.Linear(input_dims, hidden_dims)
        self.layer2 = nn.Linear(hidden_dims, hidden_dims)
        self.layer3 = nn.Linear(hidden_dims, hidden_dims)
        self.layer4 = nn.Linear(hidden_dims, hidden_dims)
        self.layer5 = nn.Linear(hidden_dims, hidden_dims)
        self.layer6 = nn.Linear(hidden_dims, hidden_dims)
        self.layer7 = nn.Linear(hidden_dims, hidden_dims)
        self.output = nn.Linear(hidden_dims, output_dims)

        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, mean=0.0, std=0.05)
                nn.init.constant_(m.bias, 0.0)

    def forward(self, x):
        x = nn.Flatten()(x)
```

```
        x = self.layer1(x)
        x = nn.Sigmoid()(x)
        x = self.layer2(x)
        x = nn.Sigmoid()(x)
        x = self.layer3(x)
        x = nn.Sigmoid()(x)
        x = self.layer4(x)
        x = nn.Sigmoid()(x)
        x = self.layer5(x)
        x = nn.Sigmoid()(x)
        x = self.layer6(x)
        x = nn.Sigmoid()(x)
        x = self.layer7(x)
        x = nn.Sigmoid()(x)
        out = self.output(x)

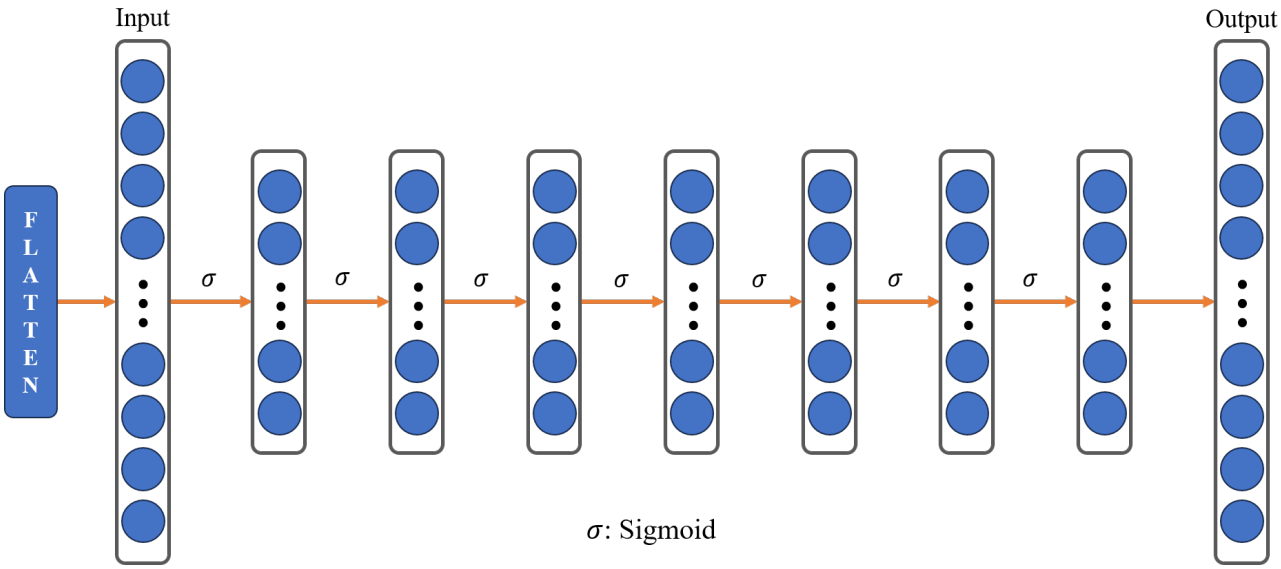
        return out

input_dims = 784
hidden_dims = 128
output_dims = 10
lr = 1e-2

model = MLP(input_dims=input_dims,
            hidden_dims=hidden_dims,
            output_dims=output_dims).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
```

Theo đó, để mô hình thể hiện được vấn đề Vanishing Gradient, ta thiết kế mô hình với số lớp ẩn là 7, mỗi lớp ẩn sẽ có 128 node, và hàm kích hoạt được sử dụng là hàm Sigmoid.



Hình 11: Minh họa kiến trúc thành phần của mô hình baseline trong project.

II.6. Huấn luyện mô hình:

Khi đã chuẩn bị đầy đủ các thành phần cần thiết bao gồm mô hình, DataLoader, hàm mất mát và thuật toán tối ưu, chúng ta có thể tiến hành huấn luyện mô hình. Quá trình huấn luyện bao gồm hai giai đoạn: train và validation. Trong mỗi epoch, mô hình được tối ưu trên tập train và đánh giá trên tập validation. Kết quả được lưu lại để phục vụ việc đánh giá và trực quan hóa sau này.

Vòng lặp huấn luyện

```
epochs = 100
train_loss_lst = []
train_acc_lst = []
val_loss_lst = []
val_acc_lst = []

for epoch in range(epochs):
    train_loss = 0.0
    train_acc = 0.0
    count = 0

    model.train()
    for X_train, y_train in train_loader:
        X_train, y_train = X_train.to(device), y_train.to(device)
        optimizer.zero_grad()
        outputs = model(X_train)
        loss = criterion(outputs, y_train)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
        train_acc += (torch.argmax(outputs, 1) == y_train).sum().item()
        count += len(y_train)

    train_loss /= len(train_loader)
    train_loss_lst.append(train_loss)
    train_acc /= count
    train_acc_lst.append(train_acc)

    val_loss = 0.0
    val_acc = 0.0
    count = 0
    model.eval()
    with torch.no_grad():
        for X_val, y_val in val_loader:
            X_val, y_val = X_val.to(device), y_val.to(device)
            outputs = model(X_val)
            loss = criterion(outputs, y_val)
            val_loss += loss.item()
            val_acc += (torch.argmax(outputs, 1) == y_val).sum().item()
            count += len(y_val)

    val_loss /= len(val_loader)
    val_loss_lst.append(val_loss)
```

```
    val_acc /= count
    val_acc_lst.append(val_acc)

    print(f"EPOCH {epoch+1}/{epochs}, Train_Loss: {train_loss:.4f}, Train_Acc: {
                                            train_acc:.4f}, Validation Loss: {val_loss
                                            :.4f}, Val_Acc: {val_acc:.4f}")
```

II.7. Trực quan hóa kết quả huấn luyện:

Sau khi hoàn tất quá trình huấn luyện, việc trực quan hóa kết quả giúp chúng ta hiểu rõ hơn về cách mô hình học và hiệu suất của nó trên các tập dữ liệu. Dưới đây là cách vẽ biểu đồ Loss và Accuracy cho cả tập train và validation.

Vẽ biểu đồ

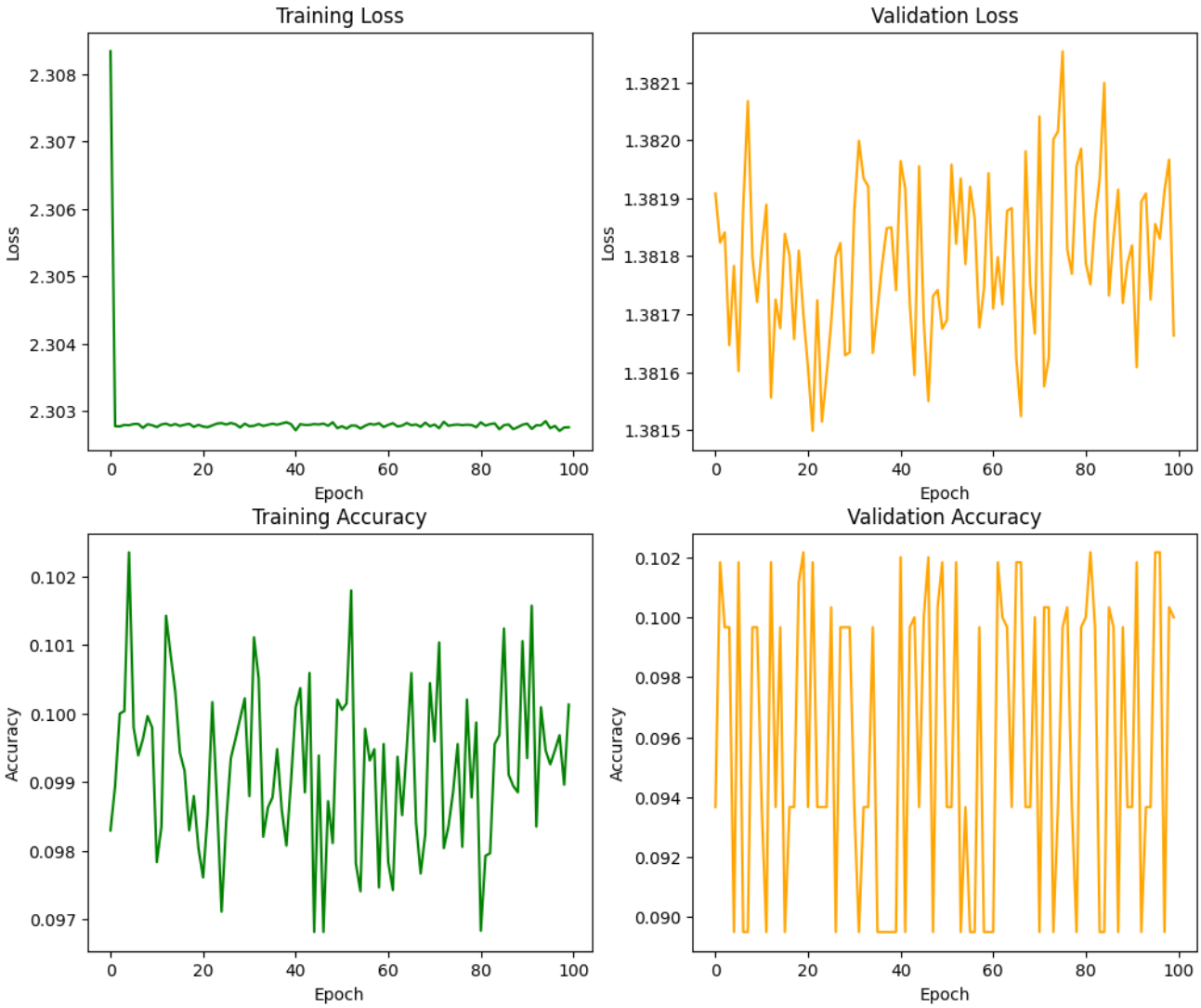
```
fig, ax = plt.subplots(2, 2, figsize=(12, 10))
ax[0, 0].plot(train_loss_lst, color='green')
ax[0, 0].set(xlabel='Epoch', ylabel='Loss')
ax[0, 0].set_title('Training Loss')

ax[0, 1].plot(val_loss_lst, color='orange')
ax[0, 1].set(xlabel='Epoch', ylabel='Loss')
ax[0, 1].set_title('Validation Loss')

ax[1, 0].plot(train_acc_lst, color='green')
ax[1, 0].set(xlabel='Epoch', ylabel='Accuracy')
ax[1, 0].set_title('Training Accuracy')

ax[1, 1].plot(val_acc_lst, color='orange')
ax[1, 1].set(xlabel='Epoch', ylabel='Accuracy')
ax[1, 1].set_title('Validation Accuracy')

plt.show()
```



Hình 12: Trực quan hóa hiệu suất mô hình baseline trong quá trình training. Có thể thấy mô hình không thể đạt sự hội tụ tốt do vướng phải hiện tượng Vanishing Gradient.

II.8. Đánh giá mô hình:

Cuối cùng, sau khi đã huấn luyện mô hình và kiểm tra trên tập validation, chúng ta cần đánh giá hiệu suất của mô hình trên tập test. Quá trình đánh giá này sử dụng độ đo Accuracy để kiểm tra khả năng phân loại của mô hình trên dữ liệu chưa từng thấy.

Đánh giá trên tập test

```
test_target = []
test_predict = []

model.eval()
with torch.no_grad():
    for X_test, y_test in test_loader:
```

```
        X_test = X_test.to(device)
        y_test = y_test.to(device)
        outputs = model(X_test)

        test_predict.append(outputs.cpu())
        test_target.append(y_test.cpu())

    test_predict = torch.cat(test_predict)
    test_target = torch.cat(test_target)
    test_acc = (torch.argmax(test_predict, 1) == test_target).sum().item() / len(
                                            test_target)

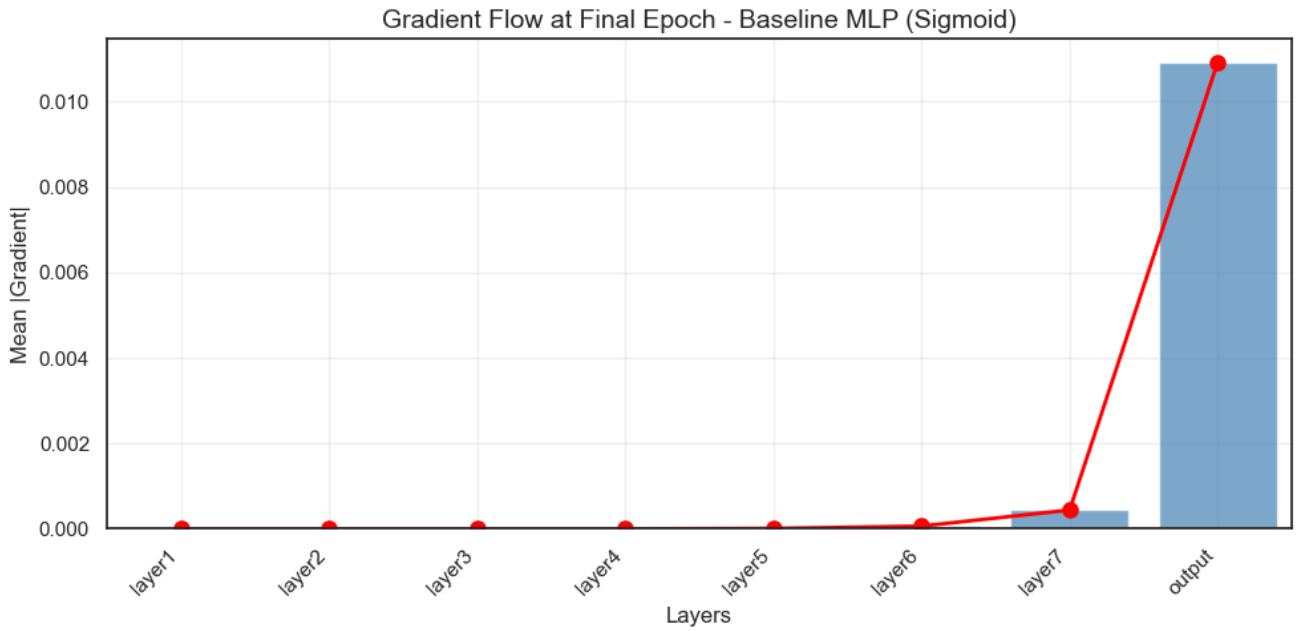
    print('Evaluation on test set:')
    print(f'Accuracy: {test_acc}')
```

Kết quả của baseline

```
Evaluation on val set:
  Accuracy: 0.0895
Evaluation on test set:
  Accuracy: 0.1
```

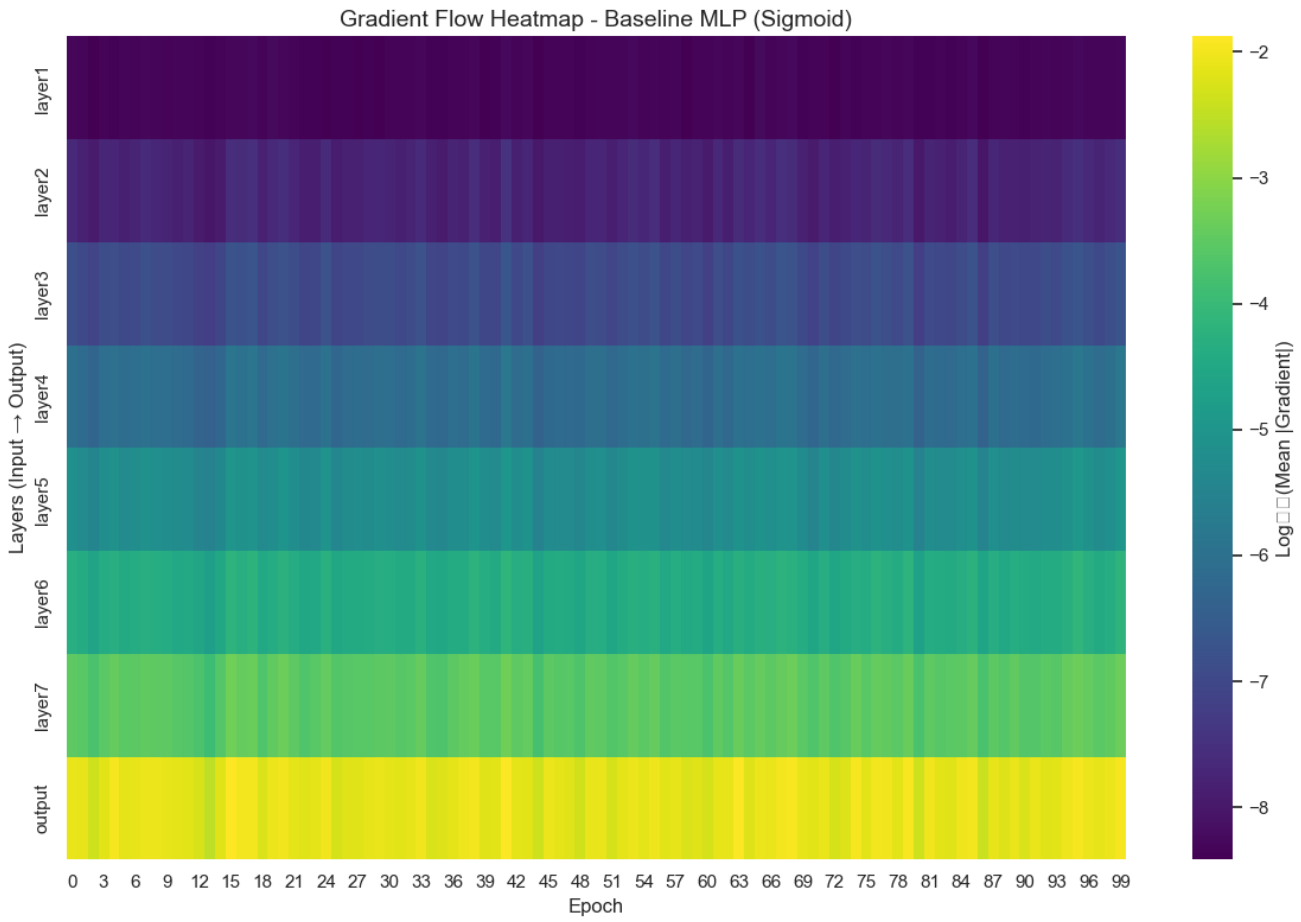
II.9. Tại sao mô hình không học được?

Mặc dù chúng ta đã thiết lập một mạng MLP tiêu chuẩn, kết quả huấn luyện cho thấy mô hình học rất chậm và gần như không hội tụ. Để hiểu rõ nguyên nhân sâu xa, ta cần "nhìn sâu" vào bên trong mạng lưới thông qua giá trị Gradient của từng lớp. Dưới đây là hai biểu đồ trực quan hóa dòng chảy của Gradient (Gradient Flow) minh chứng cho sự thất bại này.



Hình 13: Gradient Flow tại Epoch cuối cùng. Trục hoành là các lớp từ Input (trái) đến Output (phải). Đường màu đỏ biểu diễn giá trị trung bình của Gradient. Có thể thấy Gradient tại Output layer khá lớn, nhưng giảm đột ngột và tiệm cận 0 ngay từ layer 6 trở về trước.

Quan sát biểu đồ dòng chảy Gradient ở trên, ta nhận thấy một hiện tượng tai hại. Tại output layer, giá trị gradient khá lớn (xấp xỉ 0.01). Điều này chứng tỏ loss function đang hoạt động bình thường và tạo ra tín hiệu sửa lỗi ở đầu ra.Tuy nhiên, khi tín hiệu này lan truyền ngược về layer 7, nó giảm đi đáng kể. Đặc biệt, từ layer 6 trở về layer 1, đường màu đỏ gần như nằm bẹp xuống trục hoành ($y = 0$). Điều này đồng nghĩa với việc các lớp này hầu như không nhận được tín hiệu cập nhật nào. Trọng số của chúng đứng yên, và mạng nơ-ron thực chất chỉ đang học bằng 1-2 lớp cuối cùng (giống như một mô hình nông - Shallow Network).



Hình 14: Heatmap mô tả độ lớn Gradient qua toàn bộ quá trình huấn luyện (100 Epochs). Màu vàng thể hiện Gradient lớn, màu tím đậm thể hiện Gradient xấp xỉ 0 (thang đo Logarit). Các lớp đầu (Layer 1-3) chìm trong bóng tối suốt quá trình train.

Biểu đồ Heatmap củng cố thêm nhận định trên với góc nhìn theo thời gian qua từng epoch. Điều này xác nhận rằng hiện tượng Vanishing Gradient không phải là nhất thời, mà là một trạng thái dường như "tê liệt" vĩnh viễn của mô hình. Các lớp chịu trách nhiệm trích xuất đặc trưng cơ bản ở đầu mạng đã bị triệt tiêu gradient.

Ghi chú: Hiện tượng bão hòa đạo hàm

Việc sử dụng hàm Sigmoid (có đạo hàm luôn nhỏ hơn 0.25) chồng lên nhau qua 7 lớp đã khiến tích các đạo hàm tiến về 0 cực nhanh theo quy tắc chuỗi. Đây chính là động lực để ta thử nghiệm các giải pháp thay thế (như ReLU, Batch Norm, Skip Connection) ở phần tiếp theo của project.

Dựa trên baseline model đã xây dựng, chúng ta sẽ triển khai code cài đặt các phương pháp cải tiến nhằm giảm thiểu vấn đề Vanishing Gradient trong baseline model. Mỗi phương pháp các bạn cần phải chạy lại toàn bộ chương trình huấn luyện sau khi áp dụng cải tiến để thấy được kết quả. Song vì hầu hết các phần code sẽ tương đồng với code baseline, code cho các giải pháp sẽ chỉ đề cập đến đoạn cần cập nhật. Các bạn cần đọc kỹ code baseline và chỉnh sửa sao cho phù hợp nhất.

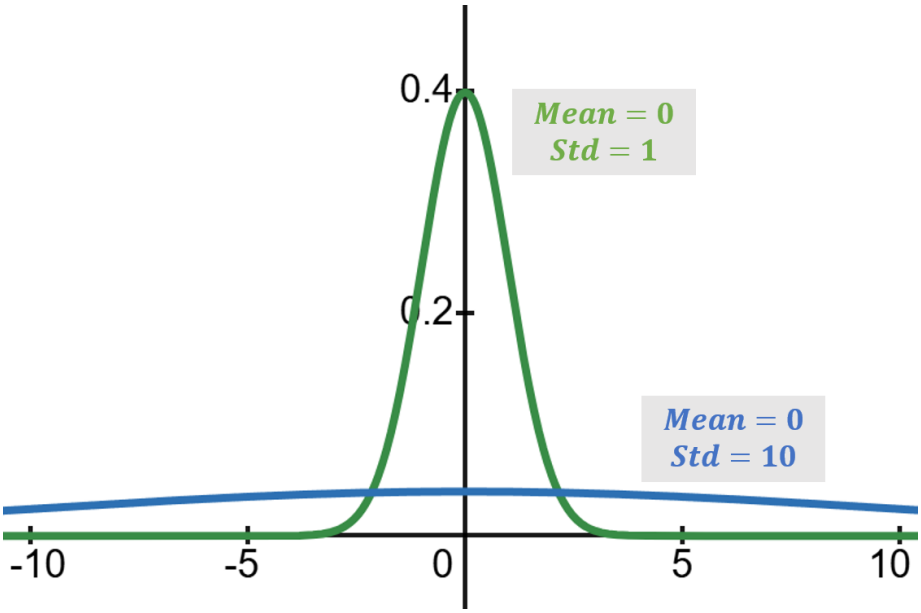
III. Các cải tiến

Sau khi đã thiết lập baseline, chúng ta sẽ lần lượt áp dụng các kỹ thuật để khắc phục vấn đề Vanishing Gradient.

III.1. Weight Increasing

Kỹ thuật này tập trung vào việc khởi tạo trọng số ban đầu (thích hợp) bằng cách thử nghiệm đơn giản với các giá trị lớn hơn nhằm tăng giá trị gradient trong các bước lan truyền ngược. Khởi tạo weight phù hợp giúp hạn chế việc gradient bị triệt tiêu qua các lớp, đặc biệt khi sử dụng các hàm kích hoạt dễ bão hòa như Sigmoid.

Tại đây, chúng ta triển khai ý tưởng này bằng cách khởi tạo trọng số với phân phối chuẩn (normal distribution) và sử dụng hai thiết lập khác nhau để đánh giá tác động: một với độ lệch chuẩn (*std*) là 1.0 và một với *std* là 10.0.



Hình 15: Normal Distribution với mean bằng 0 và standard deviation bằng 1 và 10.

III.1.1. Trường hợp $std = 1.0$

Khởi tạo trọng số với std

```python
class MLP(nn.Module):
    def __init__(self, input_dims, hidden_dims, output_dims):
        super(MLP, self).__init__()
        self.layer1 = nn.Linear(input_dims, hidden_dims)
        self.layer2 = nn.Linear(hidden_dims, hidden_dims)
        self.layer3 = nn.Linear(hidden_dims, hidden_dims)
        self.layer4 = nn.Linear(hidden_dims, hidden_dims)
        self.layer5 = nn.Linear(hidden_dims, hidden_dims)
        self.layer6 = nn.Linear(hidden_dims, hidden_dims)
```

```
        self.layer7 = nn.Linear(hidden_dims, hidden_dims)
        self.output = nn.Linear(hidden_dims, output_dims)

        for module in self.modules():
            if isinstance(module, nn.Linear):
                nn.init.normal_(module.weight, mean=0.0, std=1.0)
                nn.init.constant_(module.bias, 0.0)

    def forward(self, x):
        x = nn.Flatten()(x)
        x = self.layer1(x)
        x = nn.Sigmoid()(x)
        x = self.layer2(x)
        x = nn.Sigmoid()(x)
        x = self.layer3(x)
        x = nn.Sigmoid()(x)
        x = self.layer4(x)
        x = nn.Sigmoid()(x)
        x = self.layer5(x)
        x = nn.Sigmoid()(x)
        x = self.layer6(x)
        x = nn.Sigmoid()(x)
        x = self.layer7(x)
        x = nn.Sigmoid()(x)
        out = self.output(x)

        return out
```

III.1.2. Trường hợp $std = 10.0$

Các bạn thay đổi vào vị trí khởi tạo trọng số của đoạn code trên như sau:

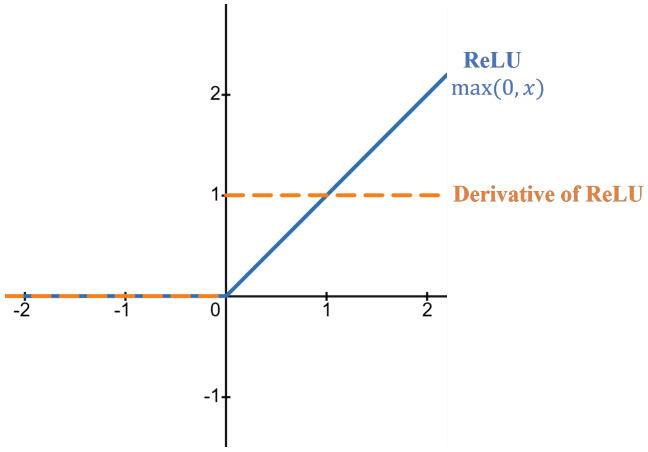
Khởi tạo trọng số với std

```
for module in self.modules():
    if isinstance(module, nn.Linear):
        nn.init.normal_(module.weight, mean=0.0, std=10.0)
        nn.init.constant_(module.bias, 0.0)
```

III.2. Better Activation

Kỹ thuật này liên quan đến việc thay đổi hàm kích hoạt trong mạng thành các hàm tiên tiến hơn. Những hàm này có khả năng hạn chế hiện tượng bão hòa, giúp gradient duy trì giá trị hợp lý qua các lớp mạng.

Tại đây, chúng ta triển khai ý tưởng này bằng cách thay thế tất cả các hàm Sigmoid trong mạng baseline bằng ReLU. Ngoài ra, trọng số của các lớp Linear cũng được khởi tạo với độ lệch chuẩn nhỏ hơn ($std = 0.05$) để phù hợp với tính chất của ReLU.



Hình 16: Đồ thị hàm ReLU.

Sử dụng hàm kích hoạt ReLU

```python
class MLP(nn.Module):
    def __init__(self, input_dims, hidden_dims, output_dims):
        super(MLP, self).__init__()
        self.layer1 = nn.Linear(input_dims, hidden_dims)
        self.layer2 = nn.Linear(hidden_dims, hidden_dims)
        self.layer3 = nn.Linear(hidden_dims, hidden_dims)
        self.layer4 = nn.Linear(hidden_dims, hidden_dims)
        self.layer5 = nn.Linear(hidden_dims, hidden_dims)
        self.layer6 = nn.Linear(hidden_dims, hidden_dims)
        self.layer7 = nn.Linear(hidden_dims, hidden_dims)
        self.output = nn.Linear(hidden_dims, output_dims)

        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, mean=0.0, std=0.05)
                nn.init.constant_(m.bias, 0.0)

    def forward(self, x):
        x = nn.Flatten()(x)
        x = self.layer1(x)
        x = nn.ReLU()(x)
        x = self.layer2(x)
        x = nn.ReLU()(x)
        x = self.layer3(x)
        x = nn.ReLU()(x)
        x = self.layer4(x)
        x = nn.ReLU()(x)
        x = self.layer5(x)
        x = nn.ReLU()(x)
        x = self.layer6(x)
        x = nn.ReLU()(x)
        x = self.layer7(x)
        x = nn.ReLU()(x)
        out = self.output(x)

        return out
```

III.3. Better Optimizer

Lựa chọn thuật toán tối ưu phù hợp có thể giúp cải thiện vấn đề Vanishing Gradient bằng cách điều chỉnh gradient linh hoạt hơn trong quá trình lan truyền ngược.

Tại đây, chúng ta thay SGD bằng Adam, một thuật toán hiện đại với khả năng tự động điều chỉnh tốc độ học, để kiểm tra hiệu quả cải thiện của nó trong việc giảm thiểu Vanishing Gradient.

Sử dụng Optimizer Adam

```
input_dims = 784
hidden_dims = 128
output_dims = 10

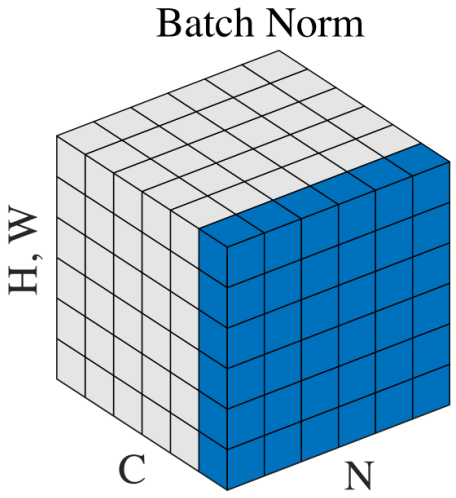
# Baseline network
model = MLP(input_dims=input_dims,
            hidden_dims=hidden_dims,
            output_dims=output_dims).to(device)

criterion = nn.CrossEntropyLoss()

lr = 1e-3
optimizer = optim.Adam(model.parameters(), lr=lr)
```

III.4. Normalize Inside Network

Batch Normalization là một kỹ thuật phổ biến để giữ các đầu vào của từng lớp mạng trong khoảng ổn định, giúp gradient không bị triệt tiêu hoặc phóng đại. Kỹ thuật này cũng góp phần tăng tốc độ hội tụ và cải thiện khả năng tổng quát hóa của mô hình. Theo đó, ta sẽ thử nghiệm hai kỹ thuật normalization:



Hình 17: Minh hoạ Batch Normalization. Nguồn ảnh: Link.

III.4.1. Sử dụng module nn.BatchNorm1d

Trong phần này, chúng ta sẽ khai báo thêm `nn.BatchNorm1d` và đặt sau mỗi lớp Linear trong mạng baseline. Batch Normalization giúp giảm thiểu nguy cơ bão hòa và duy trì gradient ổn định qua các lớp.

Tích hợp BatchNorm1d

```
class MLP(nn.Module):
    def __init__(self, input_dims, hidden_dims, output_dims):
        super(MLP, self).__init__()
        self.hidden_dims = hidden_dims
        self.layer1 = nn.Linear(input_dims, hidden_dims)
        self.bn1 = nn.BatchNorm1d(hidden_dims)
        self.layer2 = nn.Linear(hidden_dims, hidden_dims)
        self.bn2 = nn.BatchNorm1d(hidden_dims)
        self.layer3 = nn.Linear(hidden_dims, hidden_dims)
        self.bn3 = nn.BatchNorm1d(hidden_dims)
        self.layer4 = nn.Linear(hidden_dims, hidden_dims)
        self.bn4 = nn.BatchNorm1d(hidden_dims)
        self.layer5 = nn.Linear(hidden_dims, hidden_dims)
        self.bn5 = nn.BatchNorm1d(hidden_dims)
        self.layer6 = nn.Linear(hidden_dims, hidden_dims)
        self.bn6 = nn.BatchNorm1d(hidden_dims)
        self.layer7 = nn.Linear(hidden_dims, hidden_dims)
        self.bn7 = nn.BatchNorm1d(hidden_dims)
        self.output = nn.Linear(hidden_dims, output_dims)

        for module in self.modules():
            if isinstance(module, nn.Linear):
                nn.init.normal_(module.weight, mean=0.0, std=0.05)
                nn.init.constant_(module.bias, 0.0)

    def forward(self, x):
        x = nn.Flatten()(x)

        x = self.layer1(x)
        x = self.bn1(x)
        x = nn.Sigmoid()(x)
        x = self.layer2(x)
        x = self.bn2(x)
        x = nn.Sigmoid()(x)
        x = self.layer3(x)
        x = self.bn3(x)
        x = nn.Sigmoid()(x)
        x = self.layer4(x)
        x = self.bn4(x)
        x = nn.Sigmoid()(x)
        x = self.layer5(x)
        x = self.bn5(x)
        x = nn.Sigmoid()(x)
        x = self.layer6(x)
        x = self.bn6(x)
        x = nn.Sigmoid()(x)
```

```
        x = self.layer7(x)
        x = self.bn7(x)
        x = nn.Sigmoid()(x)

        out = self.output(x)
        return out
```

III.4.2. Customized Normalization Layer

Ngoài ra, chúng ta có thể tự xây dựng một class normalization thủ công như sau:

Tự định nghĩa lớp Normalization

```
class MyNormalization(nn.Module):
    def __init__(self):
        super().__init__()

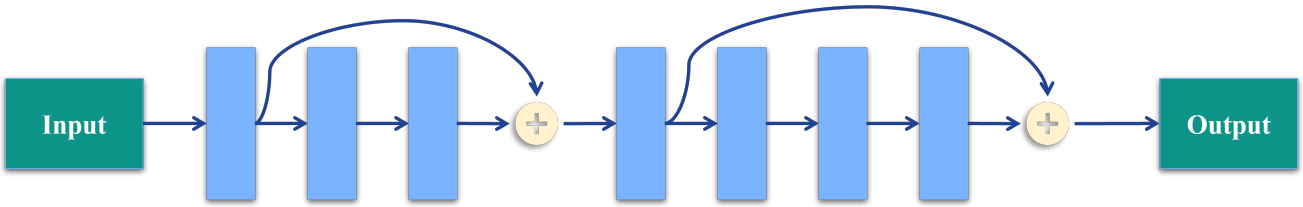
    def forward(self, x):
        mean = torch.mean(x)
        std  = torch.std(x)
        return (x - mean) / std
```

Sau đó, các bạn hãy thay thế các vị trí được áp dụng `nn.BatchNorm1d()` bằng `MyNormalization()`.

III.5. Skip Connection

Được giới thiệu trong kiến trúc của mạng ResNet, kỹ thuật này giúp gradient có thể lan truyền trực tiếp qua các lớp mà không bị triệt tiêu. Điều này không chỉ cải thiện khả năng hội tụ mà còn giữ được thông tin từ các tầng trước đó.

Tại đây, chúng ta triển khai skip connections bằng cách cộng đầu ra của các lớp trước với các lớp tiếp theo tại một số điểm trong mạng.



Hình 18: Minh họa về kỹ thuật skip connection được ứng dụng trong baseline network.

Cài đặt Skip Connection

```
class MLP(nn.Module):
    def __init__(self, input_dims, hidden_dims, output_dims):
        super(MLP, self).__init__()
        self.layer1 = nn.Linear(input_dims, hidden_dims)
        self.layer2 = nn.Linear(hidden_dims, hidden_dims)
```

```
        self.layer3 = nn.Linear(hidden_dims, hidden_dims)
        self.layer4 = nn.Linear(hidden_dims, hidden_dims)
        self.layer5 = nn.Linear(hidden_dims, hidden_dims)
        self.layer6 = nn.Linear(hidden_dims, hidden_dims)
        self.layer7 = nn.Linear(hidden_dims, hidden_dims)
        self.output = nn.Linear(hidden_dims, output_dims)

        for module in self.modules():
            if isinstance(module, nn.Linear):
                nn.init.normal_(module.weight, mean=0.0, std=0.05)
                nn.init.constant_(module.bias, 0.0)

    def forward(self, x):
        x = nn.Flatten()(x)
        x = self.layer1(x)
        x = nn.Sigmoid()(x)
        skip = x

        x = self.layer2(x)
        x = nn.Sigmoid()(x)
        x = self.layer3(x)
        x = nn.Sigmoid()(x)
        x = skip + x

        x = self.layer4(x)
        x = nn.Sigmoid()(x)
        skip = x

        x = self.layer5(x)
        x = nn.Sigmoid()(x)
        x = self.layer6(x)
        x = nn.Sigmoid()(x)
        x = self.layer7(x)
        x = nn.Sigmoid()(x)
        x = skip + x

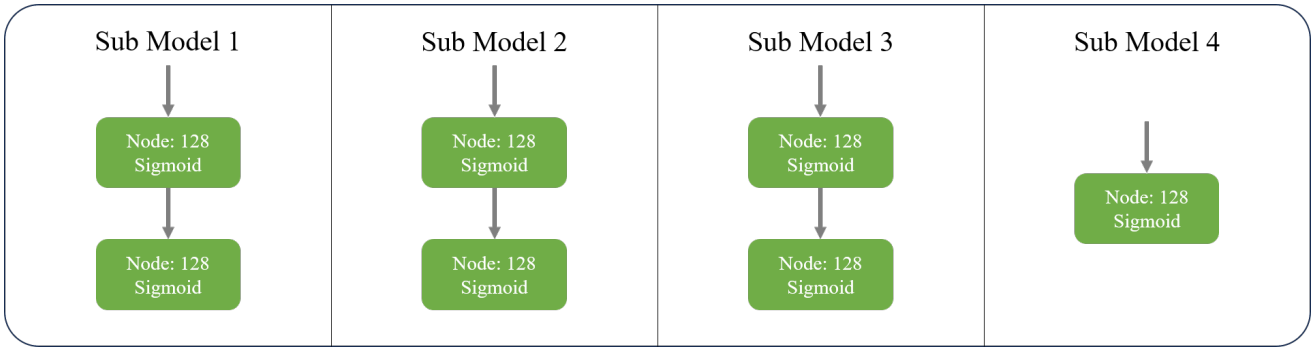
        out = self.output(x)

        return out
```

III.6. Train Layers Separately (Fine-tuning)

Trong các mạng rất sâu, việc huấn luyện toàn bộ các lớp có thể dẫn đến hiệu suất thấp do vấn đề Vanishing Gradient. Bằng cách chỉ huấn luyện một số lớp cụ thể, mô hình có thể tập trung vào việc học các đặc trưng quan trọng hơn mà không bị ảnh hưởng bởi các lớp sâu hơn.

Kỹ thuật này được triển khai bằng cách xây dựng các mô hình nhỏ hơn tương ứng với từng số lượng lớp cần huấn luyện, sau đó tăng dần các lớp được tham gia huấn luyện.



Hình 19: Mô hình ban đầu gồm 7 layer sẽ được chia thành 4 mô hình con với số lượng layer 2:2:2:1.

III.6.1. Xây dựng và Khởi tạo các module thành phần

Các thành phần mạng (module) nhỏ hơn được xây dựng tương ứng với số lớp mong muốn. Chúng ta bắt đầu với `MLP_1layer` (1 lớp) và `MLP_2layers` (2 lớp).

Định nghĩa Module con

```python
class MLP_1layer(nn.Module):
    def __init__(self, input_dims, output_dims):
        super(MLP_1layer, self).__init__()
        self.layer1 = nn.Linear(input_dims, output_dims)

        for module in self.modules():
            if isinstance(module, nn.Linear):
                nn.init.normal_(module.weight, mean=0.0, std=0.05)
                nn.init.constant_(module.bias, 0.0)

    def forward(self, x):
        x = nn.Flatten()(x)
        x = self.layer1(x)
        x = nn.Sigmoid()(x)
        return x


class MLP_2layers(nn.Module):
    def __init__(self, input_dims, output_dims):
        super(MLP_2layers, self).__init__()
        self.layer1 = nn.Linear(input_dims, output_dims)
        self.layer2 = nn.Linear(output_dims, output_dims)

        for module in self.modules():
            if isinstance(module, nn.Linear):
                nn.init.normal_(module.weight, mean=0.0, std=0.05)
                nn.init.constant_(module.bias, 0.0)

    def forward(self, x):
        x = nn.Flatten()(x)
        x = self.layer1(x)
        x = nn.Sigmoid()(x)
```

```
        x = self.layer2(x)
        x = nn.Sigmoid()(x)
        return x
```

Sau đó, ta khởi tạo các instance cụ thể cho `first`, `second`, `third`, và `fourth`:

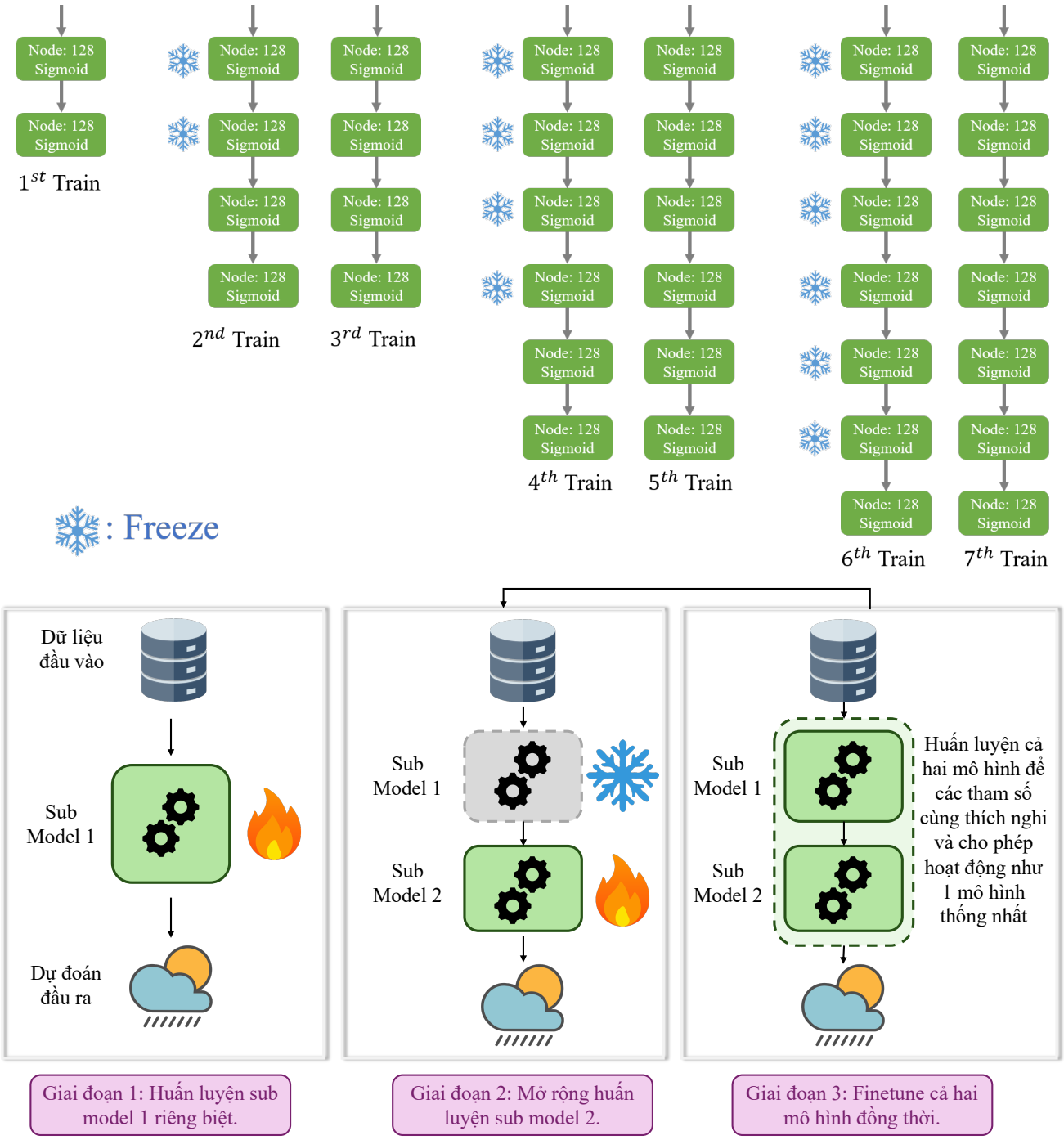
Khởi tạo Instances

```
first = MLP_2layers(input_dims=784, output_dims=128)
second = MLP_2layers(input_dims=128, output_dims=128)
third = MLP_2layers(input_dims=128, output_dims=128)
fourth = MLP_1layer(input_dims=128, output_dims=128)

lr = 1e-2
criterion = nn.CrossEntropyLoss()
```

III.6.2. Quy trình huấn luyện từng giai đoạn

Chúng ta sẽ thực hiện huấn luyện lần lượt từng mô hình con. Sau khi một mô hình con đã huấn luyện xong, ta đóng băng trọng số đã train của mô hình đó, kết nối mô hình con tiếp theo và tiếp tục quá trình training.



Hình 20: Quy trình huấn luyện tuần tự từng phần.

1. Giai đoạn 1 - Huấn luyện chỉ với thành phần đầu tiên: Chỉ thành phần `first` tham gia vào quá trình huấn luyện, nối với lớp output.

Giai đoạn 1

```
model = nn.Sequential(
    first,
    nn.Linear(128, 10)
```

```
).to(device)

optimizer = optim.SGD(model.parameters(), lr=lr)
# Training code ...
```

2. Giai đoạn 2 - Thêm thành phần thứ hai: Thêm `second`, đóng băng `first`.

Giai đoạn 2

```
for param in first.parameters():
    param.requires_grad = False

model = nn.Sequential(
    first,
    second,
    nn.Linear(128, 10)
).to(device)

optimizer = optim.SGD(model.parameters(), lr=lr)
# Training code ...
```

3. Giai đoạn 3 - Cập nhật toàn bộ: Mở khóa `first`, huấn luyện lại toàn bộ mạng hiện có.

Giai đoạn 3

```
for param in first.parameters():
    param.requires_grad = True

model = nn.Sequential(
    first,
    second,
    nn.Linear(128, 10)
).to(device)

optimizer = optim.SGD(model.parameters(), lr=lr)
# Training code ...
```

4. Giai đoạn 4 - Thêm thành phần thứ ba: Thêm `third`, đóng băng `first` và `second`.

Giai đoạn 4

```
for param in first.parameters():
    param.requires_grad = False
for param in second.parameters():
    param.requires_grad = False

model = nn.Sequential(
    first,
    second,
```

```
    third,
    nn.Linear(128, 10)
).to(device)

optimizer = optim.SGD(model.parameters(), lr=lr)
# Training code ...
```

5. Giai đoạn 5 - Cập nhật toàn bộ: Mở khóa tất cả các phần hiện có.

Giai đoạn 5

```
for param in first.parameters():
    param.requires_grad = True
for param in second.parameters():
    param.requires_grad = True

model = nn.Sequential(
    first,
    second,
    third,
    nn.Linear(128, 10)
).to(device)

optimizer = optim.SGD(model.parameters(), lr=lr)
# Training code ...
```

6. Giai đoạn 6 - Thêm thành phần thứ tư: Thêm `fourth`, đóng băng các phần trước.

Giai đoạn 6

```
for param in first.parameters():
    param.requires_grad = False
for param in second.parameters():
    param.requires_grad = False
for param in third.parameters():
    param.requires_grad = False

model = nn.Sequential(
    first,
    second,
    third,
    fourth,
    nn.Linear(128, 10)
).to(device)

optimizer = optim.SGD(model.parameters(), lr=lr)
# Training code ...
```

7. Giai đoạn 7 - Mở khóa toàn bộ thành phần: Cuối cùng, mở khóa tất cả các lớp và huấn luyện toàn bộ mô hình (Fine-tuning).

Giai đoạn 7

```
for param in first.parameters():
    param.requires_grad = True
for param in second.parameters():
    param.requires_grad = True
for param in third.parameters():
    param.requires_grad = True

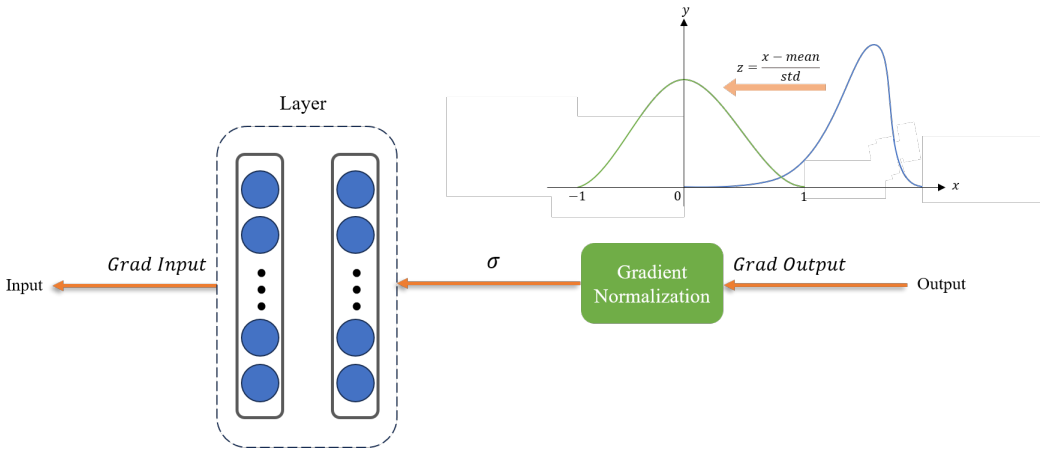
model = nn.Sequential(
    first,
    second,
    third,
    fourth,
    nn.Linear(128, 10)
).to(device)

optimizer = optim.SGD(model.parameters(), lr=lr)
# Training code ...
```

III.7. Gradient Normalization

Đây là kỹ thuật chuẩn hóa gradient trong quá trình lan truyền ngược. Kỹ thuật này đảm bảo gradient được duy trì trong một phạm vi hợp lý, tránh việc chúng trở nên quá nhỏ hoặc quá lớn, từ đó giúp cải thiện quá trình học của các lớp sâu hơn trong mạng.

Tại đây, chúng ta cài đặt một lớp `GradientNormalizationLayer`, sử dụng cơ chế `autograd` của PyTorch để chuẩn hóa gradient trong giai đoạn lan truyền ngược.



Hình 21: Minh họa về kỹ thuật Gradient Normalization.

Cài đặt Gradient Normalization

```
# Custom Gradient Normalization Layer
class GradientNormalization(torch.autograd.Function):
    @staticmethod
    def forward(ctx, input):
```

```
        # Forward pass: pass input unchanged
        ctx.save_for_backward(input)
        return input

    @staticmethod
    def backward(ctx, grad_output):
        # Normalize the gradient
        mean = torch.mean(grad_output)
        std = torch.std(grad_output)
        grad_input = (grad_output - mean) / (std + 1e-6)  # Avoid division by zero
        return grad_input

# Wrapper Module for GradientNormalization
class GradientNormalizationLayer(nn.Module):
    def __init__(self):
        super(GradientNormalizationLayer, self).__init__()

    def forward(self, x):
        return GradientNormalization.apply(x)
```

Sau định nghĩa class trên, chúng ta có thể tích hợp lớp `GradientNormalizationLayer` sau các lớp Linear để đảm bảo gradient được chuẩn hóa trước khi lan truyền ngược qua từng lớp.

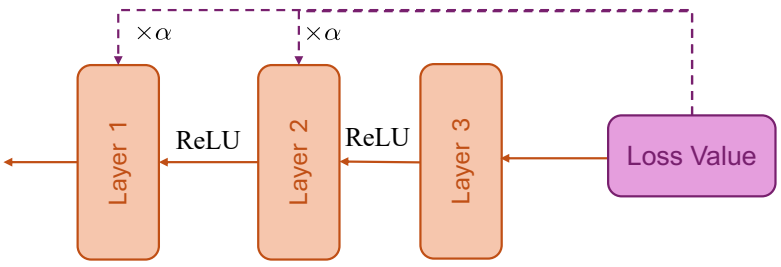
III.8. Shortcut Back

Phương pháp trong paper gốc. Shortcut Back được đề xuất cho Spiking Neural Network tại NeurIPS 2024. Trong quá trình huấn luyện, phương pháp thêm các nhánh phụ nối các tầng trung gian với đầu ra. Nhờ đó, gradient từ hàm mất mát có thêm đường truyền về các tầng nông. Các nhánh phụ được bỏ khi suy luận nên không làm tăng chi phí tính toán lúc sử dụng mô hình.

Gradient tại các tầng nông có đóng góp từ hai đường:

- Đường chính: Gradient lan truyền ngược qua các tầng theo quy tắc chuỗi (chain rule).
- Nhánh phụ: Gradient từ hàm mất mát đi qua nhánh nối với tầng trung gian, giúp tín hiệu đến các tầng nông mà không phải đi qua toàn bộ mạng.

Paper sử dụng một hệ số cân bằng để điều chỉnh đóng góp của các nhánh phụ. Ở giai đoạn đầu, các nhánh phụ được chú trọng hơn để hỗ trợ việc học ở tầng nông. Về cuối quá trình huấn luyện, trọng số của chúng giảm dần để mô hình tập trung vào đường chính.



Hình 22: Sơ đồ minh họa ý tưởng Shortcut Back trong paper gốc. Các đường tắt trong hình không được triển khai trong ví dụ MLP bên dưới.

Ví dụ triển khai cho MLP trong project. Ví dụ này lấy cảm hứng từ mục tiêu tăng cường gradient của paper nhưng dùng cách triển khai đơn giản hơn. Code không thêm nhánh phụ từ các tầng trung gian đến đầu ra. Thay vào đó, tại mỗi tầng ẩn, backward hook nhận gradient g đang lan truyền ngược và trả về

$$g' = (1 + 0{,}5\alpha)g.$$

Hệ số α giảm dần theo epoch. Theo cách đánh số tầng trong code hiện tại, tầng gần đầu ra có α lớn hơn tầng gần đầu vào. Hook chỉ nhân gradient hiện có; nó không tạo luồng gradient thứ hai từ hàm mất mát.

Dưới đây là đoạn mã cốt lõi của lớp `ShortcutBackLayer` được dùng trong ví dụ MLP.

ShortcutBackLayer Implementation

```python
# Gradient-rescaling layer (historical class name retained)
class ShortcutBackLayer(nn.Module):
    """
    Scale the existing incoming gradient with a backward hook.
    No additional gradient path or loss signal is created.
    This simplified experiment does not implement the spiking-network method
    in "Take A Shortcut Back" (NeurIPS 2024).
    """
    def __init__(self, layer_idx, num_layers, alpha_init=1.0):
        super(ShortcutBackLayer, self).__init__()
        self.layer_idx = layer_idx
        self.num_layers = num_layers
        # Higher layer indices receive larger alpha values
        self.alpha = alpha_init * (layer_idx + 1) / num_layers
        self.register_buffer('grad_scale', torch.tensor(1.0))

    def forward(self, x):
        if self.training:
            # Register a hook that scales the existing gradient
            x.register_hook(self.shortcut_backward_hook)
        return x

    def shortcut_backward_hook(self, grad):
        """
        Multiply the incoming gradient by 1 + 0.5 * alpha.
        A zero incoming gradient remains zero.
        """
        # Rescale the same gradient; this is not normalization or a new path.
        enhanced_grad = grad * (1.0 + self.alpha * 0.5)
        return enhanced_grad

    def update_alpha(self, epoch, max_epochs):
        """
        Decrease alpha with the existing exponential schedule.
        This reduces the extra gradient scaling over training.
        """
        initial_alpha_base = 1.0 * (self.layer_idx + 1) / self.num_layers
        final_alpha_base = 0.1 * (self.layer_idx + 1) / self.num_layers

        # Exponential decay
```

```
        self.alpha = initial_alpha_base * (final_alpha_base / initial_alpha_base) ** (
                                            epoch / max(max_epochs, 1))
        return self.alpha
```

ShortcutBackMLP Implementation

```
class ShortcutBackMLP(nn.Module):
    """
    MLP with training-time gradient-rescaling hooks.

    Also uses ReLU, BatchNorm, and Kaiming initialization, unlike the baseline.
    The hooks create no additional path from the loss.
    Evaluation uses this model's forward architecture without registering hooks.
    """
    def __init__(self, input_dims, hidden_dims, output_dims, alpha_init=1.0):
        super(ShortcutBackMLP, self).__init__()

        # Create layers using ModuleList for better organization
        self.num_hidden_layers = 7
        self.input_layer = nn.Linear(input_dims, hidden_dims)

        self.hidden_layers = nn.ModuleList([
            nn.Linear(hidden_dims, hidden_dims) for _ in range(self.num_hidden_layers -
                                                1)
        ])

        self.output_layer = nn.Linear(hidden_dims, output_dims)

        # Gradient-rescaling layers for each hidden layer
        self.shortcut_back_layers = nn.ModuleList([
            ShortcutBackLayer(i, self.num_hidden_layers, alpha_init)
            for i in range(self.num_hidden_layers)
        ])

        # Batch normalization for better training
        self.batch_norm_layers = nn.ModuleList([
            nn.BatchNorm1d(hidden_dims) for _ in range(self.num_hidden_layers)
        ])

        # Initialize weights
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight, mode='fan_in', nonlinearity='relu')
                nn.init.constant_(m.bias, 0.0)

    def forward(self, x):
        x = nn.Flatten()(x)

        # Input layer with ReLU and batch norm
        x = self.input_layer(x)
        x = self.batch_norm_layers[0](x)
        x = nn.ReLU()(x)
```

```
        x = self.shortcut_back_layers[0](x)

        # Hidden layers with ReLU and batch norm
        for i, hidden_layer in enumerate(self.hidden_layers):
            x = hidden_layer(x)
            x = self.batch_norm_layers[i + 1](x)
            x = nn.ReLU()(x)
            x = self.shortcut_back_layers[i + 1](x)

        # Output layer (no activation, handled by CrossEntropyLoss)
        x = self.output_layer(x)
        return x

    def update_alphas(self, epoch, max_epochs):
        """
        Update alpha coefficients for all gradient-rescaling layers.
        Called once per epoch during training.
        """
        alphas = []
        for shortcut_layer in self.shortcut_back_layers:
            alpha = shortcut_layer.update_alpha(epoch, max_epochs)
            alphas.append(alpha)
        return alphas
```

Kết quả của 8 bước cải tiến

```
Evaluation on val set:
    Accuracy: 0.87
Evaluation on test set:
    Accuracy: 0.8706
```

III.9. Progressive Training

Một cách tiếp cận khác để giải quyết vấn đề Vanishing Gradient trong các mạng rất sâu là "Progressive Training" (Huấn luyện tăng dần). Thay vì huấn luyện toàn bộ mạng sâu ngay từ đầu (nơi gradient dễ bị triệt tiêu khi lan truyền về các lớp đầu), chúng ta bắt đầu với một mạng nông (chỉ vài lớp hoạt động) và dần dần "mở khóa" (unfreeze) hoặc thêm các lớp tiếp theo trong quá trình huấn luyện.

Kỹ thuật này hoạt động dựa trên nguyên lý:

- Giai đoạn đầu: Mạng nông giúp gradient lan truyền mạnh mẽ, các lớp đầu tiên học được các đặc trưng cơ bản một cách nhanh chóng.
- Giai đoạn sau: Khi các lớp đầu đã ổn định, ta nới rộng độ sâu của mạng. Lúc này, tín hiệu gradient từ hàm loss không cần phải gồng gánh việc huấn luyện từ con số 0 cho các lớp đầu nữa (vốn đã được pre-train ở giai đoạn trước), giúp quá trình hội tụ ổn định hơn.

Dưới đây là cách triển khai lớp `ProgressiveMLP` cho phép thay đổi độ sâu động (dynamic depth) trong quá trình forward:

ProgressiveMLP Implementation

```python
class ProgressiveMLP(nn.Module):
    def __init__(self, input_dims, hidden_dims, output_dims, start_layers=2):
        super(ProgressiveMLP, self).__init__()
        self.num_hidden_layers = 7
        self.active_layers = start_layers

        # Define all layers using ModuleList
        self.layers = nn.ModuleList([
            nn.Linear(input_dims, hidden_dims),
            nn.Linear(hidden_dims, hidden_dims),
            nn.Linear(hidden_dims, hidden_dims),
            nn.Linear(hidden_dims, hidden_dims),
            nn.Linear(hidden_dims, hidden_dims),
            nn.Linear(hidden_dims, hidden_dims),
            nn.Linear(hidden_dims, hidden_dims)
        ])
        self.output = nn.Linear(hidden_dims, output_dims)

        # Freeze layers deeper than start_layers initially
        self.freeze_inactive_layers()

    def freeze_inactive_layers(self):
        for i in range(self.active_layers, self.num_hidden_layers):
            for param in self.layers[i].parameters():
                param.requires_grad = False

    def unfreeze_next_layer(self, layer_idx):
        if layer_idx < len(self.layers):
            for param in self.layers[layer_idx].parameters():
                param.requires_grad = True

    def forward(self, x):
        x = nn.Flatten()(x)
        # Only pass through active layers
        for i in range(self.active_layers):
            x = self.layers[i](x)
            x = nn.Sigmoid()(x)
        out = self.output(x)
        return out
```

Để quản lý việc nới rộng mạng, ta sử dụng một `ProgressiveTrainer`. Trainer này sẽ theo dõi số epoch và tự động mở khóa lớp tiếp theo sau một khoảng thời gian nhất định (ví dụ: mỗi 10 epochs). Đặc biệt, khi một lớp mới được thêm vào, ta áp dụng kỹ thuật "Warmup Learning Rate" cho riêng lớp đó để tránh làm hỏng các trọng số đã học được của mạng trước đó.

ProgressiveTrainer Logic

```python
class ProgressiveTrainer:
    def __init__(self, model, optimizer, base_lr, unfreeze_interval=10):
        self.model = model
```

```
        self.optimizer = optimizer
        self.base_lr = base_lr
        self.unfreeze_interval = unfreeze_interval

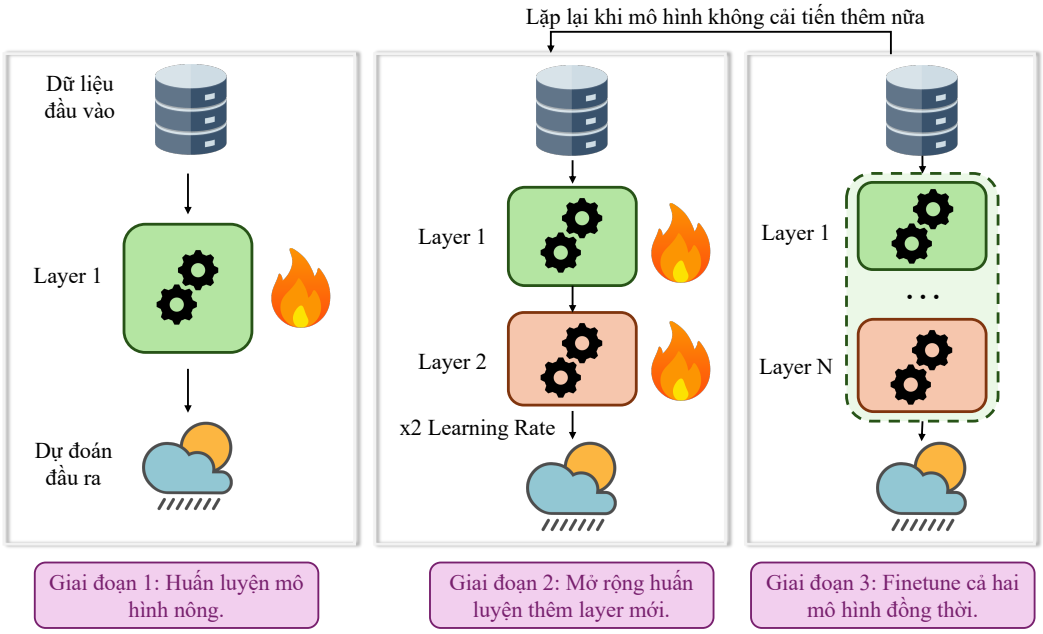
    def should_unfreeze_next_layer(self, epoch):
        # Check if we reached max depth
        if self.model.active_layers >= self.model.num_hidden_layers:
            return False
        # Unfreeze every 'unfreeze_interval' epochs
        if (epoch + 1) \% self.unfreeze_interval == 0:
            return True
        return False

    def unfreeze_next_layer(self, epoch):
        layer_idx = self.model.active_layers
        self.model.unfreeze_next_layer(layer_idx)

        # Add new parameters to optimizer with higher LR (Warmup)
        new_params = self.model.get_newly_unfrozen_params(layer_idx)
        warmup_lr = self.base_lr * 2.0 # Boost LR for new layer

        self.optimizer.add_param_group({
            'params': new_params,
            'lr': warmup_lr,
            'warmup_layer': True
        })
        self.model.active_layers += 1
        print(f"Layer {layer_idx} unfrozen. Active: {self.model.active_layers}")
```





Hình 23: Quy trình huấn luyện tuần tự từng giai đoạn.

Kết quả thực nghiệm trên tập dữ liệu FashionMNIST cho thấy sự cải thiện rõ rệt so với baseline thuần túy:

Kết quả Progressive Training

```
Evaluation on val set:
    Accuracy: 0.8386
Evaluation on test set:
```

```
    Accuracy: 0.8304
```

Như vậy, chúng ta đã tìm hiểu và áp dụng một số kỹ thuật nhằm khắc phục vấn đề Vanishing Gradient trong mạng MLP. Mặc dù vẫn còn nhiều kỹ thuật khác, mục tiêu chính của project này là làm rõ sự tồn tại và tác động của vấn đề Vanishing Gradient, đồng thời minh họa các giải pháp cơ bản để xử lý vấn đề này.

IV. Tài liệu tham khảo
Phụ lục

1. **Code:** Các file code được đề cập trong bài có thể được tải tại đây.
2. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng 3-4 tiếng.



Hình 24: Hình ảnh group facebook AIO Q&A.